

## Concurrencia y Paralelismo

### Bloque I: Concurrencia

Junio 2022

1. *Gestión de salida [2 puntos]* Un sistema gestiona la salida a disco a través de un único thread que recibe peticiones del resto de threads del sistema. Cuando un thread quiere realizar una operación de escritura, envía una petición a través de una cola. Para simplificar el problema, asumimos que la cola es segura frente a accesos concurrentes (no es necesario bloquear el acceso a ella).

```

struct request {
    request_info inf;
    ...
};

// Funciones ya implementadas en otros modulos del sistema
struct request *remove_request();
void insert_request(struct request *r);

void submit_request(struct request *r) {
    insert_request(r);
}

void wait_till_done(struct request *r) {
    ...
}

void *io_thread(void *ptr) {
    while(1) {
        struct request *r;

        r = remove_request();
        do_io(r->inf);
    }
}

```

El thread que gestiona las escrituras se ejecuta en `io_thread`. Los threads que quieren realizar una operación llaman a `submit_request`, e insertan una `struct request` en la cola. Esa estructura es retirada de la cola por el thread de escritura, y se comparte entre esos dos threads.

Implemente la función `wait_till_done`, y modifique `io_thread` y `submit_request` para que los threads que han enviado una petición puedan llamarla para esperar hasta que ésta se haya completado. Si la escritura ya se completó la llamada debería volver inmediatamente. Puede añadir más campos a la estructura `request` si lo necesita.

```

int pthread_cond_broadcast(pthread_cond_t *cond);
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
    void *(*fun) (void *), void *arg);
int pthread_join(pthread_t thread, void **retval);
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
int pthread_cond_init(pthread_cond_t *cond, pthread_condattr_t *cond_attr);
int pthread_cond_destroy(pthread_cond_t *cond);

```

## Solución

---

```
struct request {
    request_info inf;
    pthread_cond_t *request_not_done;
    pthread_mutex_t *request_m;
    int done;
};

struct request *remove_request();
void insert_request(struct request *r);

void submit_request(struct request *r) {
    r->done = 0;
    pthread_mutex_init(&r->request_m, NULL);
    pthread_cond_init(&r->request_not_done, NULL);
    insert_request(r);
}

void wait_till_done(struct request *r) {
    pthread_mutex_lock(&r->request_m);
    if(!r->done)
        pthread_cond_wait(&r->request_not_done, &r->request_m);
    pthread_mutex_unlock(&r->request_m);
}

void *io_thread(void *ptr) {
    while(1) {
        struct request *r;

        r = remove_request();
        do_io(r->inf);

        pthread_mutex_lock(&r->request_m);
        r->done = 1;
        pthread_cond_broadcast(&r->request_not_done);
        pthread_mutex_unlock(&r->request_m);
    }
}
```

---

## 2. *Threads que simulan corredores de una carrera por relevos [2 puntos]*

Crear la función `runner()` que simula un corredor en una carrera de relevos.

Condiciones a tener en cuenta:

- No pueden usarse variables globales.
- No puede hacerse espera activa.
- No pueden usarse semáforos.
- El primer (1) y el último (N) corredor son especiales.
- El primero tiene que esperar a que se de la salida, y que ésta sea válida. Si no es válida hay que repetir la salida. La variable `valid_start` indica si la salida es válida. La variable de condición `start` se lanza cada vez que hay una salida.
- Cada corredor indicará con un mensaje desde que hora está listo, a que hora recibe el testigo y a que hora termina su relevo. Usar la función `seconds_since_start()` para saber cuantos segundos han pasado desde el comienzo de la carrera.
- El último corredor de cada equipo indicará en que posición ha llegado.
- Los corredores de cada equipo tienen que correr en orden de número dentro del equipo (parámetro `id_runner`).
- Cada corredor tiene que pasar el testigo al siguiente corredor del mismo equipo. Pista, úsese un campo en la `struct team`.
- El programa tiene que funcionar incluso si un corredor se despierta en un momento que no le toque correr.
- Puede suponerse que la salida se efectúa después de que todos los corredores estén listos para empezar.
- Cada corredor tiene que ejecutar la función `run_100_meters()`, que es la que simula que ha corrido los metros que le corresponden.
- Se pueden añadir los campos que sean necesarios a las `struct global` y a la `struct team`.
- La `struct global` es la misma para todos los corredores.
- La `struct team` es la misma para todos los miembros de un equipo.
- Hay que minimizar el tiempo que pasa desde que un corredor puede correr hasta que empieza a correr.
- Añadir un comentario a cada campo de las estructuras indicando qué significa y a que valor se inicializa.

---

```
struct global {
    pthread_cond_t start; /* broadcast when race begins */
    pthread_mutex_t m;    /* protect everything on this struct */
    bool valid_start;      /* indicates if the start was good. Initialized to false */
};

struct team {
    int num_runners_team;
};

void runner(struct global *g, struct team *t, int id_runner)
{
    ...
    run_100_meters();
    ...
}

int seconds_since_start(void);
```

---

## Solución

---

```
#include <pthread.h>
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>

struct global {
    pthread_cond_t start; /* broadcast when race begins */
    pthread_mutex_t m;    /* protect everything on this struct */
    bool valid_start;     /* indicates if the start was good. Initialized to false */
    int position;         /* current position at end */
};

struct team {
    int num_runners_team;
    char *team_name;
    pthread_cond_t wait_token[]; /* index is id_runner - 1 */
    pthread_mutex_t m;
    int next;
};

void runner(struct global *g, struct team *t, int id_runner)
{
    printf("Runner %d is ready at %d\n", id_runner, seconds_since_start());
    if (id_runner == 1) {
        pthread_mutex_lock(&g->m);
        while (!g->valid_start)
            pthread_cond_wait(&g->start, &g->m);
        pthread_mutex_unlock(&g->m);
        pthread_mutex_lock(&m);
        t->next = 2;
    } else {
        pthread_mutex_lock(&t->m);
        while (t->next != id_runner) {
            pthread_cond_wait(&t->wait_token[id_runner - 2], &t->m);
        }
        t->next++;
        pthread_mutex_unlock(&t->m);
    }

    printf("Runner %d starts running at %d\n", id_runner, seconds_since_start());
    run_100_meters();
    printf("Runner %d ends running at %d\n", id_runner, seconds_since_start());

    if (id_runner == t->num_runner_team) {
        pthread_mutex_lock(&g->m);
        int pos = g->position;
        pthread_mutex_lock(&g->m);
        printf("Team %s finish in position %d\n", pos);
    } else {
        pthread_cond_signal(&t->wait_token[id_runner - 1]);
    }
}

int seconds_since_start(void)
{
    return time();
}
```

---

### 3. Procesos en anillo [1 punto]

Tenemos un sistema con un grupo de procesos distribuidos en una estructura de anillo, donde cada proceso conoce únicamente el PID del siguiente.

La implementación del sistema es la siguiente:

---

```
-module(ring).  
  
-export([start/1, init/0, ring_size/1]).  
  
start(N) ->  
    Pids = start_processes(N),  
    send_next(Pids),  
    Pids.  
  
ring_size(Pid) ->  
    ...  
  
start_processes(0) -> [];  
start_processes(N) -> [spawn(?MODULE, init, []) | start_processes(N - 1)].  
  
send_next([], []) -> ok;  
send_next([Pid | Pids], [Next_pid | Next_pids]) ->  
    Pid ! {next, Next_pid},  
    send_next(Pids, Next_pids).  
  
send_next([Pid | Pids]) ->  
    send_next([Pid | Pids], Pids ++ [Pid]).  
  
init() ->  
    receive  
        {next, Next_pid} ->  
            loop(Next_pid)  
    end.  
  
loop(Next_pid) ->  
    ...  
    loop(Next_pid).
```

---

Implemente la función `ring_size/1`, que dado el pid de uno de los procesos del anillo, devuelve el número total de procesos que forman parte del mismo. Modifique únicamente las Funciones `ring_size` y `loop`

## Solución

---

```
-module(ring).

-export([start/1, init/0, ring_size/1]).

start(N) ->
    Pids = start_processes(N),
    send_next(Pids),
    Pids.

ring_size(Pid) ->
    Pid ! {get_size, self(), Pid},
    receive
        {get_size_reply, Size} ->
            Size
    end.

start_processes(0) -> [];
start_processes(N) -> [spawn(?MODULE, init, []) | start_processes(N - 1)].

send_next([], []) -> ok;
send_next([Pid | Pids], [Next_pid | Next_pids]) ->
    Pid ! {next, Next_pid},
    send_next(Pids, Next_pids).

send_next([Pid | Pids]) ->
    send_next([Pid | Pids], Pids ++ [Pid]).

init() ->
    receive
        {next, Next_pid} ->
            loop(Next_pid)
    end.

loop(Next_pid) ->
    receive
        {get_size, From, First} ->
            Next_pid ! {go_around, From, First, 1};
        {go_around, From, self(), N} ->
            From ! {get_size_reply, N};
        {go_around, From, First, N} ->
            Next_pid ! {go_around, From, First, N+1}
    end,
    loop(Next_pid).
```

---

# EXAMEN DE CONCURRENCIA Y PARALELISMO

## Bloque II: Paralelismo

APELLIDOS: \_\_\_\_\_ NOMBRE: \_\_\_\_\_

Convocatoria ordinaria

6 de junio de 2022

**AVISO:** *No mezcléis en el mismo folio preguntas del bloque de concurrencia y del bloque de paralelismo. Las respuestas a cada bloque se entregarán por separado.*

- [2.5p] 1. **Conceptos de paralelismo.** En estos momentos de “gripalización del COVID” un determinado gobierno, más preocupado por las restricciones que por aumentar los gastos sanitarios, quiere hacer una encuesta para conocer qué uso sigue haciendo la población de las mascarillas. Para ello se desarrolla un ambicioso proyecto en el que se pretende llamar a todos los 300000 domicilios que existen en esta región de 30000km<sup>2</sup>. Cada llamada durará un minuto (tanto respondan como no).

Cabe destacar que este proyecto empieza con una tarea inicial de cuatro horas en la que el director divide el mapa de la zona en fragmentos de igual tamaño (10km<sup>2</sup>). Esta tarea solo puede ser realizada por el director ya que requiere unos conocimientos técnicos que solo posee él. Además, una vez finalizadas las encuestas, este director también necesitará otras cuatro horas para integrar y analizar los resultados (va analizando una encuesta recibida tras otra).

- (a) ¿Cuál es la máxima aceleración que se podría conseguir si invertimos muchos recursos materiales y humanos en la realización de las encuestas telefónicas? [0,5p]

- (b) De cara a la fase de las encuestas, asume lo siguiente:

- La densidad de población no es uniforme a lo largo de la región, sino que habrá fragmentos en los que haya que hacer muchas más llamadas que en otros.
- Las tareas del director consisten en coordinar las encuestas y redactar informes de los resultados, pero no realizar llamadas.
- El director no puede estar en contacto directo y constante con los trabajadores, sino que prefiere asignar todos los fragmentos al inicio y luego recoger los resultados al final.

¿Qué descomposición y asignación de tareas utilizarías? Razona tu respuesta. [0,5p]

- (c) Utilizando la descomposición y asignación del apartado anterior asume que:

- Se contrata a tres trabajadores además del director.
- La asignación consigue un balanceo perfecto de la carga de trabajo. Es decir, los tres trabajadores acaban haciendo el mismo número de encuestas (100000).
- Antes de empezar con las encuestas el director necesita tener una reunión de 15 minutos con cada trabajador (por separado) donde le explica lo que tiene que hacer y le asigna sus fragmentos de mapa (estos 15 minutos de reunión serán necesarios sea cual sea el número de fragmentos a entregar).
- La recolección de datos se hace de forma telemática e instantánea.

Calcula las siguientes métricas teniendo en cuenta el trabajo del director [1,0p]:

- i. Tiempo paralelo

- ii. Speedup
- iii. Eficiencia
- iv. Coste
- v. Sobrecarga

(d) ¿Qué optimización se te ocurre si el director pudiese estar disponible en todo momento, pero no está dispuesto a realizar llamadas? [0,5p]

[2.5p] 2. **Diseño de algoritmos paralelos.** Un periódico mantiene una matriz  $A$  de tamaño  $C \times P$  donde  $A_{ij}$  mide el nivel de implicación de la persona  $j$  en el caso de corrupción  $i$ . Así mismo, recibe periódicamente dos matrices  $X$  e  $Y$  de tamaños  $P \times M$  y  $P \times N$ , respectivamente, donde el elemento de la fila  $j$  y columna  $k$  mide el nivel de asociación de la persona  $j$  a la entidad  $k$ . La diferencia entre ambas radica en que la línea editorial denuncia la corrupción de las entidades de la matriz  $X$  e ignora las de la matriz  $Y$ . De hecho, para decidir en qué se va a centrar el periódico<sup>1</sup>, se calcula un índice de interés  $I_i$  sobre cada caso de corrupción  $I_i$  como

```

 $I_i = 0;$ 
for( $j = 0;$   $j < P;$   $j++$ )
    for( $k = 0;$   $k < M;$   $k++$ )
         $I_i = I_i + A_{ij} \times X_{jk};$ 
    for( $k = 0;$   $k < N;$   $k++$ )
         $I_i = I_i - A_{ij} \times Y_{jk};$ 

```

y se eligen los casos con índices más altos. Todas las matrices son de tipo `double` y se dispone de un único servidor con disco duro suficiente para almacenar las matrices, el cual está conectado a otros 9 ordenadores, los cuales no tienen conexión entre sí, de forma que todo mensaje involucra al servidor. Todos tienen la misma CPU, de forma que toda operación en punto flotante o copia de un dato requiere un ciclo de CPU mientras que la transmisión de un dato por la red requiere 3. El sistema tiene una versión limitada de MPI que sólo soporta comunicaciones bloqueantes. Además las colectivas no finalizan en ningún procesador hasta que todos los mensajes se han completado, y el proceso 0 se ejecuta en el servidor. Se desea paralelizar el cómputo del interés de los casos siguiendo el esquema:

- 1) `read(&A,&X,&Y,&C,&P,&M,&N)` en el servidor
- 2) paralelizar el cómputo de  $I$  entre los 10 ordenadores
- 3) `print(I, C)` en el servidor

donde `read(double** A, double** X, double** Y, int* C, int* P, int* M, int* N)` reserva y lee del disco las matrices de entrada descritas en el primer párrafo así como sus tamaños  $C$ ,  $P$ ,  $M$  y  $N$ , mientras que `print(double *I, int N)` imprime el vector  $I$  de  $N$  componentes computados de acuerdo al algoritmo. Supón que todas las dimensiones de las matrices y vectores son divisibles por 10.

---

<sup>1</sup>Todo parecido entre este enunciado y el funcionamiento de cualquier medio de comunicación es mera coincidencia.



```

int MPI_Send(void *buf, int count, MPI_Datatype datatype,
             int dest, int tag, MPI_Comm comm)
int MPI_Recv(void *buf, int count, MPI_Datatype datatype,
             int source, int tag, MPI_Comm comm, MPI_Status *status)
int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype,
              int root, MPI_Comm comm)
int MPI_Scatter(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
               void *recvbuf, int recvcnt, MPI_Datatype recvtype,
               int root, MPI_Comm comm)
int MPI_Gather(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
               void *recvbuf, int recvcnt, MPI_Datatype recvtype,
               int root, MPI_Comm comm)
int MPI_Reduce(void *sendbuf, void *recvbuf, int count,
               MPI_Datatype datatype, MPI_Op op,
               int root, MPI_Comm comm)

```

- (a) Describe la paralelización más directa de este problema explicando de qué se haría cargo cada procesador, cómo se distribuirían y/o replicarían las matrices, y el tipo de asignación de tareas. [0.5p]
- (b) Escribe el código MPI asociado a esa solución de la forma más sencilla posible (asumiendo directamente 10 procesos, usando colectivas siempre que sea posible, etc.) [1p]
- (c) Calcula para el programa del apartado b) el número de datos que el servidor envía y recibe a/de cada uno de los otros ordenadores para  $C=50$ ,  $P=200$ ,  $M=20$  y  $N=20$ . [0.5p]
- (d) En esta situación en la que el número total  $P$  de implicados es mucho más elevado que el de casos  $C$  y de entidades  $M$  y  $N$ , ¿sería mejor otra estrategia de paralelización? Si la hay, descríbela claramente, usando de apoyo un dibujo ilustrativo y justifícala calculando el número de datos transmitido entre el servidor y cada otro procesador para el tamaño del problema propuesto en el apartado anterior así como la variación en término de computaciones si la hay. Si no la hay, justifica también tu respuesta de la misma forma y viendo que es peor que la ya implementada. [0.5p]

## SOLUCIÓN

1. (a) Las tareas iniciales (descomponer el mapa) y final (integrar y analizar datos) necesitan 240 minutos cada una, mientras que las llamadas tardan un total de 300000 minutos. Eso significa que la fracción de tiempo intrínsecamente secuencial es  $\frac{240+240}{300000+240+240} = 0.0016$ . Por tanto, de acuerdo a la Ley de Amdahl la aceleración máxima es:  $\frac{1}{0.0016} = 625$ .
  - (b) Descomposición de dominio donde hacer todas las llamadas de cada fragmento del mapa es una tarea. En este caso se necesita una asignación estática que asigne todos los fragmentos al inicio, porque luego el director no estará disponible. Dentro de las estáticas vale la pena usar una cíclica porque es de esperar que la carga de trabajo esté agrupada por fragmentos cercanos (por ejemplo, núcleos urbanos) y así la cíclica distribuirá mejor dicha carga.
  - (c) i. El tiempo paralelo es la suma de:
    - El tiempo inicial de fragmentación hecho solo por el director: 240 minutos.
    - El tiempo de las tres reuniones. Aunque el primero de los trabajadores podrá empezar mientras se hacen el resto de reuniones, el último será el cuello de botella porque tendrá que esperar a que esté libre el director. 45 minutos.
    - El tiempo de realizar las encuestas en paralelo: 100000 minutos
    - El tiempo de integrar y analizar los resultados: 240 minutos.Esto implica que el tiempo paralelo es de:  $240 + 100000 + 45 + 240 = 100525$  minutos
  - ii. El speedup es  $\frac{300480}{100525} = 2.99$
  - iii. La eficiencia es  $\frac{3}{4} = 0.75$
  - iv. El coste es  $100525 * 4 = 402100$  minutos
  - v. La Sobrecarga es  $402100 - 300480 = 101620$  minutos
  - (d) Una posible mejora en ese caso sería que los trabajadores fuesen mandando los resultados de cada fragmento según acaben todas las encuestas de dicho fragmento. El director podría ir las integrando y analizando poco a poco mientras se sigue con las encuestas y así dejar casi al mínimo el tiempo final de análisis.  
Una asignación maestro-esclavo no sería una buena opción porque para asignar cada fragmento necesita una reunión presencial bastante larga (sobre 15 minutos) con los trabajadores y, por tanto, estarían parando constantemente de hacer llamadas.
2. (a) Haría que cada proceso calculase  $C/10$  elementos de  $I$  usando una descomposición de dominio de las matrices con una asignación estática. Cada proceso recibiría un bloque de  $C/10$  filas consecutivas de la matriz  $A$  y las matrices  $X$  e  $Y$  enteras, con lo que éstas serían replicadas. Entonces aplicaría el algoritmo de cómputo del interés sólo al rango de filas de  $A$  recibidas. El vector  $I$  global se obtendría en el servidor mediante una operación gather. La asignación de tareas es estática.

(b)

```

double *A, *X, *Y, *I;
int rank, C, P, M, N, i, j, k;

MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
if(!rank) {
    read(&A, &X, &Y, &C, &P, &M, &N);
}
MPI_Bcast(&C, 1, MPI_INT, 0, MPI_COMM_WORLD);
MPI_Bcast(&P, 1, MPI_INT, 0, MPI_COMM_WORLD);
MPI_Bcast(&M, 1, MPI_INT, 0, MPI_COMM_WORLD);
MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD);
if(rank) {
    A = (double *)malloc(sizeof(double) * C/10 * P);
    X = (double *)malloc(sizeof(double) * P * M);
    Y = (double *)malloc(sizeof(double) * P * N);
}
I = (double *)malloc(sizeof(double) * C);
MPI_Scatter(A, C/10 * P, MPI_DOUBLE,
    rank ? A : MPI_IN_PLACE, C/10 * P, MPI_DOUBLE, 0, MPI_COMM_WORLD);
MPI_Bcast(X, P * M, MPI_DOUBLE, 0, MPI_COMM_WORLD);
MPI_Bcast(Y, P * N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
for(i = 0; i < C/10; i++) {
    I[i]=0;
    for(j = 0; j < P; j++) {
        for(k=0; k < M; k++)
            I[i] += A[i * P + j] * X[j * M + k];
        for(k=0; k < N; k++)
            I[i] -= A[i * P + j] * Y[j * N + k];
    }
}
MPI_Gather(rank ? I : MPI_IN_PLACE, C/10, MPI_DOUBLE,
    I, C/10, MPI_DOUBLE, 0, MPI_COMM_WORLD);
if(!rank)
    print(I, C);
MPI_Finalize();

```

- (c) Enviamos un total de  $4 + P \times (C/10 + M + N)$  datos a cada proceso y recibimos  $C/10$  de él, totalizando  $4 + 200 \times (50/10 + 20 + 20) + 50/10 = 9009$  datos.
- (d) En clase hemos visto que lo mejor es particionar las dimensiones más grandes, en este caso  $P$ . Eso implicaría partir  $A$  por bloques de columnas y  $X$  e  $Y$  por bloques de filas. Así cada proceso obtendría un vector parcial  $I$  de  $C$  elementos que mediría el impacto en el  $I$  global del grupo de personas que le hubiera tocado a cada proceso, y habría que sumarlos todos para obtener el  $I$  final. De esta forma, el servidor enviaría a cada proceso  $4 + P/10 \times (C + M + N)$  datos y recibiría  $C$  para reducir, totalizando en este caso  $4 + 200/10 \times (50 + 20 + 20) + 50 = 1854$  datos.

Esta descomposición requiere un poco más de computación al necesitar una reducción global que reduce 10 datos (9 sumas) por cada uno de los  $C$  elementos finales de  $I$ , totalizando  $9 \times 50 = 450$  sumas en nuestro caso. Como el enunciado indica que cada computación requiere un ciclo y el envío de cada dato 3, claramente esta opción es mucho mejor a pesar de la reducción adicional.

